

Volcado de resultados por pantalla

Quitar el punto y coma tras una asignación → vuelca resultado por pantalla

```
>> a = pi
```

```
a =
```

```
3.1416
```

Usar la función disp()

```
>> e=exp(1);
```

```
>> disp('El número e vale'); disp(e)
```

El resultado es

```
2.7183
```

En ambos casos si queremos más decimales:

```
>> format long; disp(e) → 2.718281828459045
```

```
>> format short; disp(e) → 2.7183
```

Volcado formateado de datos (fprintf)

```
fprintf("-----", a, b, ...);
```

Cadena de texto conteniendo:

- Texto → se vuelca tal cual por pantalla.
- Comandos de "impresión" como \n (salto de línea) o \t (TAB)
- Descriptores (%d, %f, %e) describiendo cómo volcar cada una de las variables (entero, número real con decimales, notación científica, ...)

Pueden completarse con modificadores:

%4d → volcar como entero dejando 4 espacios

%.6f → formato zz.xxxxxx con 6 decimales después del punto

%.3e → notación científica con 3 decimales en la mantisa.

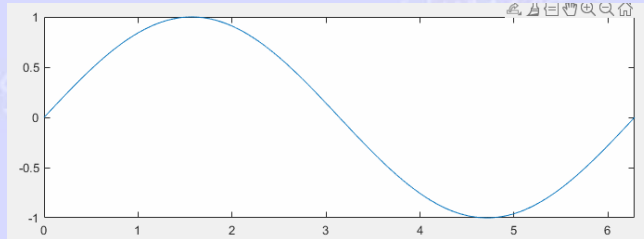
Gráficos en MATLAB

Comando fundamental es `plot(x,y)`

Recibe un vector con los datos de las $\{x\}$ y otro con los datos de las $\{y\}$

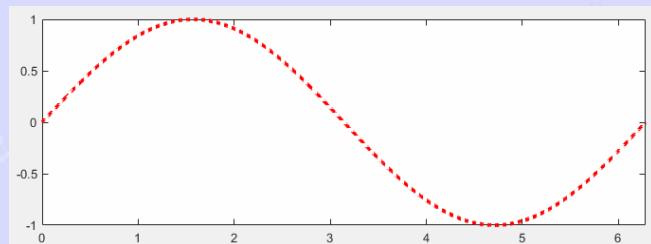
Hace una gráfica de los puntos (x_k, y_k) y los une por líneas.

```
x=(0:0.001:2*pi); y=sin(x); plot(x,y);
```



Pueden añadirse modificadores para cambiar colores, tipo de línea, ...

```
plot(x,y,'r:',Linewidth=3);
```



Varias gráficas simultaneamente

```
x=linspace(0,2*pi,500); f1=sin(x); f2=cos(x);
```

a) En ventanas distintas: `figure; plot(x,f1);`
`figure; plot(x,f2)`

b) En diferentes ejes de la misma ventana usando `subplot()`

```
subplot(2,1,1); plot(x,f1);  
subplot(2,1,2); plot(x,f2)
```

c) Superpuestas:

Con un solo comando `plot`: `plot(x,f1,x,f2);`

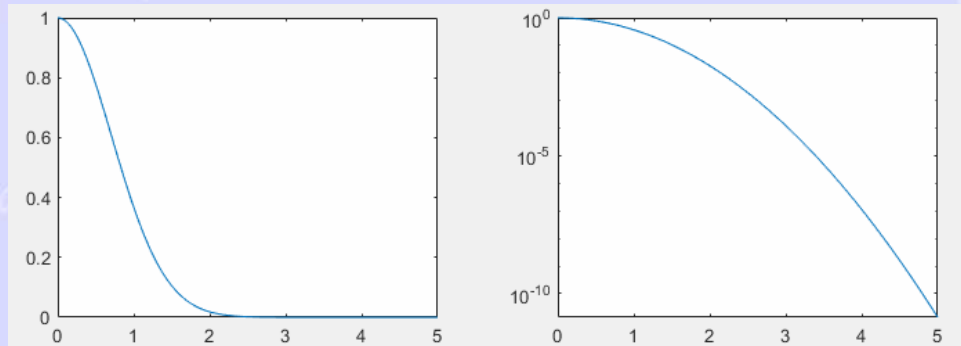
Con sucesivos comandos `plot()` y el comando `hold on`; `hold off`

```
plot(x,f1); hold on;  
plot(x,f2);  
plot(...)  
hold off;
```

Otros comandos para gráficos

a) Escala logarítmica en el eje de las Y's: `semilogy(x,y)`

```
x=(0:0.01:5); y=exp(-x.^2);  
subplot(121); plot(x,y); subplot(122); semilogy(x,y);
```



b) Escala logarítmica en el eje de las X's: `semilogx(x,y)`

c) Escala logarítmica en ambos ejes: `loglog(x,y)`

Ejecución Condicional

```
if condicion  
    codigo;  
end
```

```
if condicion  
    codigo1;  
else  
    codigo2;  
end
```

Condiciones:

Resultados de operadores de comparación: `>`, `>=`, `<`, `<=`, `==`, `~=`

Podemos combinarlas con operadores lógicos: AND (`&`)

OR (`|`)

NOT (`~`)

```
arg=2;  
switch arg  
    case 1,    codigo1;  
    case 2,    codigo2;  
    case 3,    codigo1;  
end
```

Bucles for

```
for k=1:10 % k=lista de valores
    fprintf('%2d al cuadrado es %3d\n',k,k^2);
end
```

El código entre **for** y **end** se repite usando en cada caso un valor distinto para k sacado de la lista especificada.

Se puede especificar un salto mayor o menor de 1, negativo, ..

```
for k=0:3:10, ... end
for k=1:0.5:10, ... end
for k=10:-1:1, ... end
for k=[1 3 9 4], ... end
```

El comando **continue** dentro de un bucle **for** hace que se pase al siguiente valor del bucle sin completar el código del bucle.

El comando **break** dentro de cualquier bucle hace que termine el bucle.

Bucles while

```
while (condicion)
    ...
end
```

El código entre **while**, **end** se repite mientras se cumpla la condición (o condiciones) especificadas

```
k=1;
while (k<10)
    codigo;
    k=k+1;
end
```

```
k=1; err=1;
while (k<10) & (err>0.0001)
    codigo % que previsiblemente
           % disminuye el error
    k=k+1;
end
```

Aunque es posible el uso de un **break**, no tiene mucho sentido dentro de un **while** (podríamos integrar esa condición en las condiciones iniciales).

Funciones en MATLAB

- Permiten dividir un proyecto complicado en tareas más sencillas.
- Aíslan trozos de código entre sí: una función solo ve las variables que se le pasan como argumentos de entrada y al terminar solo devuelve sus/s argumentos de salida.

```
function res=fun(x)  
% Código que hace operaciones con el valor recibido en  
% la variable x introduciendo el resultado en res  
end
```

- Puede recibir múltiples argumentos de entrada o salida:

```
function [c,d]=fun2(a,b)  
% recibe los valores a,b y opera con ellos guardando  
% los resultados que quiere devolver en c y d.  
end
```

Formas de crear estas funciones

- a) Aisladas (sin ningún código adicional), en un fichero .m propio **con el mismo nombre** de la función:

```
function f=fun(x)
    f=sin(x.^2);
end
```

—————> Guardado como fun.m

Puede llamarse desde cualquier script o desde la línea de comandos:

```
>> x=(0:0.01:pi); plot(x,fun(x));
```

- b) En un script .m o .mlx: sólo son visibles dentro de ese programa

```
function f=fun(x)
    f=sin(x.^2);
end
```

—————> Guardado como ejer2.m

```
x=(0:0.01:1); plot(x,fun(x));
```

Pueden añadirse varias de estas funciones auxiliares en un único script.

Formas de crear estas funciones

- c) Crearla (dentro de un script o desde la línea de comandos) con un comando de creación de un "puntero" a función

```
fun = @(x) sin(x.^2);
```

—————> Dentro de un script o
en la línea de comandos

Mientras no se borre con un clear puede usarse en un script o desde la línea de comandos de la misma forma que las funciones de antes:

```
fun = @(x) sin(x.^2);
x=(0:0.01:1); plot(x,fun(x));
```

Esta forma tiene algunas limitaciones:

- La función debe poder escribirse con una sola expresión.
- No puede tener más de un argumento de salida.